

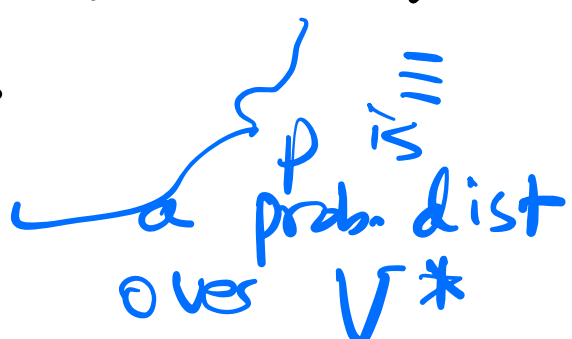
Lecture 1 : Language modeling & Markov models

V = vocab, finite

Sentence = $x_1 x_2 \dots x_n$

$$x_i \in V \quad \forall 1 \leq i \leq n$$

Def A language model is a vocab V and
 $p: V^* \rightarrow [0,1]$ s.t.

- 1) for all $\langle x_1, \dots, x_n \rangle \in V^*$ $p(x_1, \dots, x_n) \geq 0$
 - 2) $\sum_{\langle x_1, \dots, x_n \rangle \in V^*} p(x_1, \dots, x_n) = 1$
-  p is a prob. dist over V^*

Training

Corpus C = set of sentences

Def A learning algorithm for language modeling is an algo: $\mathcal{C} \rightarrow H$

$\mathcal{C}$: class of corpora

H : some class of language models

skip determining V for now (say it's fixed)

Most naive learning algo:

$$\text{let } p(x_1, \dots, x_n) := \frac{c(x_1, \dots, x_n)}{m}$$

times $\langle x_1, \dots, x_n \rangle$ occurs in C
 $\leftarrow |C| \cdot \# \text{sent}$

why bad?

- super overfit
- ignores structure of sentences
- assigns $p=0$ to any $S \notin C$

Today: Markov models

Models of sequences of r.v. $X_1, \dots, X_i, \dots$
where distribution of X_i depends only on
very recent past states
/local

let X_i = value of the i -th word in sentence

$$\begin{aligned} p(x_1, \dots, x_n) &= p(X_1 = x_1, \dots, X_n = x_n) \\ &= p(X_1 = x_1) \cdot \prod_{i=2}^n p(X_i = x_i \mid X_1 = x_1, \dots, X_{i-1} = x_{i-1}) \end{aligned}$$

Chain rule of probabilities

Markovian assumption: X_i depends only on
a few predecessors

X_j for $j < i$

eg $P(X_i = x_i \mid X_1 = x_1, \dots, X_{i-1} = x_{i-1})$

$\rightarrow = P(X_i = x_i \mid X_{i-2} = x_{i-2}, X_{i-1} = x_{i-1})$ ←
assumption

+ P is independent of i , $\forall i \neq j$

$$P(X_i = x \mid X_{i-2} = a, X_{i-1} = b)$$

$$= P(X_j = x \mid X_{j-2} = a, X_{j-1} = b)$$

For now fixed length sentences, n fixed

$$1) p(x_1, \dots, x_n) = \prod_{i=1}^n P(X_i = x_i \mid X_{i-1} = x_{i-1})$$

define $x_0 = *$

"start symbol"

- "first order" Markov model

- bigram model

$$2) p(x_1, \dots, x_n) = \prod_{i=1}^n P(X_i = x_i \mid X_{i-2} = x_{i-2}, X_{i-1} = x_{i-1})$$

define $x_0 = x_{-1} = *$ "start symbol"

- "second order" Markov
- trigram model

$$3) p(x_1, \dots, x_m) = \prod_{i=1}^m P(X_i = x_i \mid X_{i-3} = x_{i-3}, X_{i-2} = x_{i-2}, X_{i-1} = x_{i-1})$$

$$1, \dots, X_{i-1} = x_{i-1})$$

$$X_0 = X_{-1} = \dots = X_{-n+2} = *$$

(n-1)-th order Markov
"n-gram model"

Arbitrary length

set $X_n = \text{STOP}$ (special, $\notin V$) for any
valid sentence

"Las Vegas algorithm"

How to sample sentence (trigram as ex.)

$$i=1, X_0 = X_{-1} = *$$

while:

$$\text{sample } X_i \leftarrow p(X_i | X_{i-2} = X_{i-2}, X_{i-1} \neq X_{i-1})$$

if $X_i = \text{STOP}$?

break / return $X_1, \dots, X_i$

$$i := i+1$$

may not terminate, eg if $p(\text{STOP} | X_{i-1}, X_{i-2}) = 0$

Q: is p a valid prob. dist. on

$$|V|^* \times \{\text{STOP}\}$$

$$p \geq 0 \quad \checkmark \quad (\text{set } p(X_i | \dots) \geq 0)$$

$$\sum_{s \in |V|^* \times \{\text{STOP}\}} p(s) = 1$$

related

Learning algos for Markov models

eg. for bigram, "parameters" to decide
 $\forall w, v \quad q(w/v) := P(X_i = w \mid X_{i-1} = v)$

$\uparrow$ q for parametrized distribution / algorithm

$$q: |V| \times |V| \rightarrow [0, 1]$$

$\# \text{ parameters} \approx |V|^2$

- 1) STOP
- 2) constraint prob dist

$$\forall v \quad \sum_w q(w/v) = 1$$

trigram:

$$q(\underline{u, v, w}) = P(X_i = w \mid X_{i-2} = u, X_{i-1} = v)$$

(indep. of i)

$$\approx |V|^3 \text{ parameters}$$

$$|V| = 10k \text{ to } 200k +$$

encoder tokenizers

easiest

Learning algo for Markov models:

Maximum Likelihood Estimation (MLE)

= find those parameters to maximize prob. of C (training corpus)

$$q = \arg \max_q p(C) = \arg \max_q \prod_{s \in C} p(s) \\ = \{q(w/u, v)\}_{w, u, v}$$

bigram $q(w/v) = \frac{c(v, w)}{c(v)}$

$c(x_1, \dots, x_\ell)$ = count of ℓ -gram $\langle x_1, \dots, x_\ell \rangle$ in C

trigram $q(w/u, v) = \frac{c(u, v, w)}{c(u, v)}$

(Will prove that it's the MLE)

Issues? sparse data

- many $q(w/u, v) = 0 \Leftrightarrow c(u, v, w) = 0$
- if $c(u, v) = 0$, $q(w/u, v)$ undefined $\forall w$

Fixing w/ advanced n -gram models

"Smoothing" of n -gram models

MLE values

$$q_{ML}(w|u,v) = \frac{c(u,v,w)}{c(u,v)}$$

$$q_{ML}(w|v) = \frac{c(v,w)}{c(v)}$$

$$q_{ML}(w) = \frac{c(w)}{c()} \leftarrow c() = \# \text{ total words}$$

Idea #1 Linear interpolation (trigram eg)

$$q_{LI}(w|u,v) = \lambda_1 q_{ML}(w|u,v) + \lambda_2 q_{ML}(w|v) + \lambda_3 q_{ML}(w)$$

$$\lambda_1, \lambda_2, \lambda_3 \geq 0$$

choose u, v st $c(u,v) > 0$

$$\begin{aligned} \sum_w q(w|u,v) &= \sum (\lambda_1 \dots + \lambda_2 \dots + \lambda_3 \dots) \\ &= \lambda_1 \underbrace{\sum_w q_{ML}(w|u,v)}_{=1} + \lambda_2 \underbrace{\sum_w q_{ML}(w|v)}_{=1} + \lambda_3 \underbrace{\sum_w q_{ML}(w)}_{=1} \\ &= \lambda_1 + \lambda_2 + \lambda_3 \end{aligned}$$

$$q(w|u,v) \text{ is prob dist} \Leftrightarrow \sum_w q(w|u,v) = 1$$

$$\Leftrightarrow \lambda_1 + \lambda_2 + \lambda_3 = 1$$

+ rule: for $c(u,v)=0$, skip first term

$$q(w|u,v) = \frac{1}{1-\lambda_1} (\lambda_2 q_{uc}(w|v) + \lambda_3 q_{mc}(w))$$

Claim: for $\overline{c(u,v)=0}$, w/ extension above

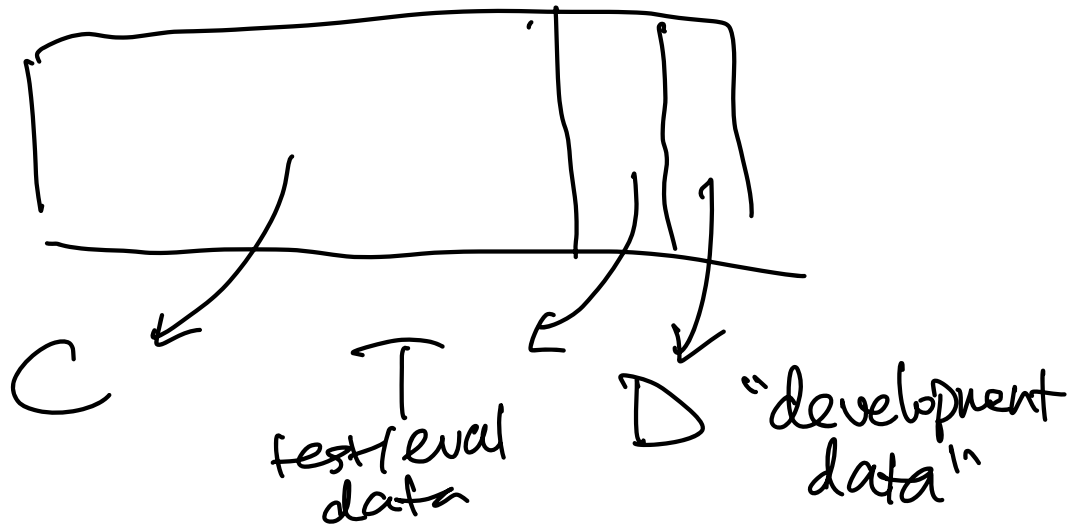
$$\sum_w q(w|u,v) = 1$$

(if v doesn't occur, $c(v)=0$, **check!!**)

$$q(w|u,v) = \frac{1}{1-\lambda_2} \frac{1}{1-\lambda_3} \lambda_3 q_{mc}(w)$$

How to choose $\lambda_1, \lambda_2, \lambda_3$?

all data



use D to optimize $\lambda_1, \lambda_2, \lambda_3 = \text{argmax } p(D|q_{mc})$

Method #2)

Discounting

bigram example $q(w|v)$

based on
C

$$c^*(v, w) = c(v, w) - \beta \quad \text{for } c(v, w) > 0$$

eg $\beta = 0.5$

for $c(v, w) > 0$, use

$$q_{\text{disc}}(w|v) = \frac{c^*(v, w)}{c(v)}$$

↑

leave "leftover mass" for unseen bigrams

$$A(v) = \{w : c(v, w) > 0\}$$

$$B(v) = \{w : c(v, w) = 0\}$$

leftover mass $\alpha(v) = 1 - \sum_{w \in A(v)} \frac{c^*(v, w)}{c(v)}$

Def $q_{\text{disc}}(w|v) = \begin{cases} \frac{c^*(v, w)}{c(v)} & w \in A(v) \\ \alpha(v) \times \frac{q_{\text{ML}}(w)}{\sum_{w \in B(v)} q_{\text{ML}}(w)} & w \in B(v) \end{cases}$

notice

$$\alpha(v) = 1 - \sum_{w \in A(v)} \frac{c(v, w) - \beta}{c(v)}$$

$$= 1 - \sum_{w \in A(v)} \frac{c(v, w)}{c(v)} + \sum_{w \in A(v)} \frac{\beta}{c(v)}$$

$$w \in A(v) \quad c(v) \quad w \in A(v) \quad c(v)$$

$$= \cancel{\frac{1}{c(v)}} + \frac{|A(v)|}{c(v)} \beta = \frac{\text{\# unique words following } v}{\text{\# count of } v} \times \beta$$

evaluating LM

Perplexity

given test/eval corpus $T = \{S_1, \dots, S_m\}$

$$\text{prob}(T) = \prod_{i=1}^m p(S_i)$$

$$S_i = X_{i,1}, \dots, X_{i,n_i} \quad n_i = \text{length of } S_i$$

$$M = \sum_{i=1}^m n_i \quad \leftarrow \text{total \# words in } T$$

$$\text{maximizing } p(T) \Leftrightarrow \text{max. } \frac{1}{M} \log_2(p(T))$$

$\hookrightarrow \log$ monotonic

$$\ell := \frac{1}{M} \sum_{i=1}^m \log(p(S_i))$$

ℓ something like empirical estimate
of entropy of random word

Def perplexity of p on test corpus I

$$:= 2^{-l}$$

good $p \Leftrightarrow$ high l

$\Leftrightarrow$ low 2^{-l}

low perplexity
= good

eg. p is Markov/unigram

$$p(w) = \frac{1}{|V|} \quad \text{completely uniform}$$

$$p(s_i) = \left(\frac{1}{|V|}\right)^{n_i}$$

$$l = \frac{1}{M} \sum_{i=1}^M n_i \log\left(\frac{1}{|V|}\right)$$

$$= \frac{1}{M} \log\left(\frac{1}{|V|}\right) \left(\sum_{i=1}^M n_i\right) = M$$

$$= \log\left(\frac{1}{|V|}\right)$$

$$\text{perp.} = 2^{-\log\left(\frac{1}{|V|}\right)} = |V|$$

[least certain model $\Rightarrow$ perp. $|V|$]

Can show $\text{perp.} \in (0, |V|]$

↑
good

↑
bad

intuition something like "effective
vocab size"